PriorityKV: Structure-Aware KV Retention with a Measurable Operating Boundary

Arush Sharma[1] and Anupam Rawat[2]
[1]Indian Institute of Technology (Indian School of Mines) Dhanbad
[2]Indian Institute of Technology Bombay

Abstract

A long agent conversation packs tool schemas, persistent identifiers, superseded instructions, and filler into one key–value (KV) cache. Every token costs the same amount of memory, but losing the wrong one costs far more than losing the wrong filler. PriorityKV uses the structure a chat transcript already exposes as a retention rule: keep schemas, constraints, and identifiers before ordinary filler.

On PriorityBench-A, a synthetic stress test built for this study, structure-aware retention passes 112 of 120 Qwen3-8B examples at a 25% keep budget, against 1 of 120 for position-blind eviction, and is statistically indistinguishable from matched-budget SnapKV and PyramidKV reimplementations at 108/120 (McNemar $p = 0.125$). The prior is strong exactly when the state worth keeping is sparse and recognizable; when that same state is instead spread through free-form text, the rule's pass rate falls to 80/120.

An external evaluation on BFCL finds where the prior stops working. There, protected content covers 98.8% of the context, far past the 25% budget, so a yes/no structural label runs out of room to discriminate, while attention-based selection still tracks FullKV. PriorityBench-A sits at the opposite extreme, with only 6.0% of tokens protected. Because this protected-to-budget ratio can be measured before generation, we use it to build ADAPT, a budget-relative blend of structure and attention with no fitted parameter. ADAPT recovers SnapKV's range on BFCL (0.129 versus 0.136, $p = 1.0$).

A separate systems study asks the same question about storage rather than eviction: role-aware INT4 placement matches uniform placement in quality, while a mixed BF16/INT4 prototype cuts KV payload to 0.719× at a 1.200–1.215× time-per-token cost. Together, the two studies give PriorityKV a retention rule with a known operating range, a cheap test for when that range ends, and an explicit memory–latency trade for storage.

1 Introduction

Consider a long agent trace whose final tool call omits a required field. The field appeared in a schema many turns earlier, but its KV entries were evicted to meet a memory budget. Byte for byte, the schema looked no different from the filler that survived; operationally, losing it invalidated the call. The same failure can resurrect an obsolete instruction or substitute the wrong persistent identifier. Figure 1 illustrates this mismatch between storage cost and failure cost.

Autoregressive inference stores prior keys and values so each decode step can reuse the prefix. The cache grows with sequence length and constrains serving capacity [1, 2]. Existing work reduces that cost through paging, streaming windows, attention-derived eviction, or quantization [3, 4, 5, 6, 7]. Agent traces expose an additional signal: their serialization already identifies message roles, tool contracts, constraints, results, and state. PriorityKV asks whether this visible structure can guide what survives.

The answer is a useful contrast. Structure succeeds when important state is sparse and recognizable, then loses its ability to discriminate when nearly all tokens carry the same protected role. That reversal exposes the paper's main result: structural retention has an operating boundary that can be measured before generation. We then test whether a soft structure–attention blend can cross that boundary gracefully. A separate BF16/INT4 study tests storage rather than eviction; Figure 2 keeps these questions distinct.

The paper follows that contrast. Sections 3–4 define the controlled benchmark and policy; Section 6 establishes the in-distribution result and its placement, budget, and model limits; Section 7 presents the external crossover, its mechanism, and ADAPT. The INT4 and systems results remain separate because eviction removes information whereas quantization keeps an approximation.

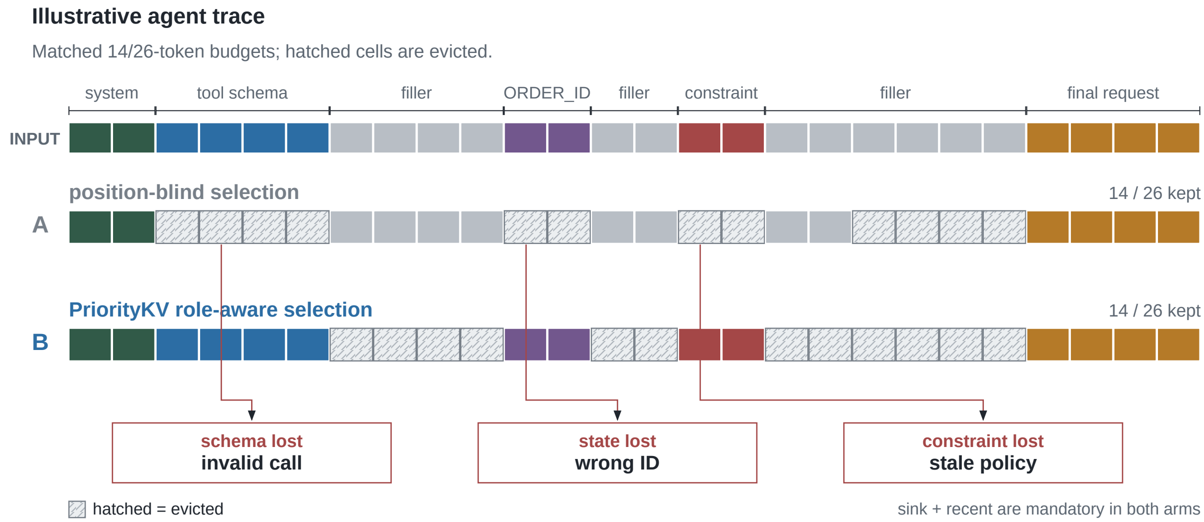

Figure 1: Equal storage cost, unequal failure cost. Two policies keep the same illustrative 14/26-token budget. Hatched cells are evicted; callouts show the resulting failure modes rather than empirical counts.

Three experiment families answer three different questions

Study	Question and comparison	Measured endpoint	Interpretation
H1	**Retention · W5, n=120** Which positions survive at k=0.25?	**112/120 vs 1/120** fixed manifests and seeds	**In-regime structural win**
H2	**INT4 quality · W3, n=240** Does role-aware placement beat uniform?	**0.8833 vs 0.8792** fixed manifests and seeds	**No quality separation**
H3	**Systems · H200, n=18** Mixed FlashInfer vs FullKV/SDPA	**0.719× / 1.200–1.215×** fixed manifests and seeds	**Payload–latency trade-off**

Figure 2: Three questions, three verdicts. Eviction reliability, precision placement, and systems performance are evaluated independently; displayed counts and ratios are measured endpoints.

This work makes four contributions.

1. We introduce PriorityBench-A, a deterministic stress test for tool-schema conformance, instruction supersession, and state recall, with placement controls and a token-level retention audit.

2. We show a large matched-budget gap over position-blind eviction on Qwen (112/120 versus 1/120), and we report honestly where that gap closes: against SnapKV-style reimplementations, and at a tighter budget on Llama.

3. We expose the external boundary on BFCL, identify oversubscription of the protected set as its mechanism, and evaluate ADAPT as a budget-relative soft prior. This yields a practical policy-selection diagnostic rather than a universal-superiority claim.

4. We independently test mixed precision, implement mixed BF16/INT4 pages with two-call FlashInfer decode, and report payload, memory, and latency without folding them into the retention claim.

2 Related Work

Paging and streaming. PagedAttention separates logical KV blocks from non-contiguous physical storage to reduce fragmentation and enable sharing; it does not choose semantic tokens to retain [2]. StreamingLLM identifies attention sinks and combines initial tokens with a recent window for streaming language modeling [3]. PriorityKV uses sink plus recent as a mandatory region and as the shape of its position-only control, not as a claim to reproduce StreamingLLM's streaming setting.

Eviction and attention-derived selection. H2O retains accumulated-attention heavy hitters together with recent tokens [4]. SnapKV uses an observation window near the end of the prompt to select clustered prefix features [5]. PyramidKV varies the cache budget across layers in response to pyramidal information funneling [6]. These methods infer importance from model behavior; PriorityKV tests application-visible message structure. The attention-baseline comparison uses repository reimplementations at matched token budgets, not the authors' reference code. In particular, H2O is a chunked SDPA reimplementation and carries an explicit fidelity caveat. A hybrid that protects structural tokens and fills the remainder with SnapKV was evaluated: it equals SnapKV on Qwen at 25% keep and is lower than both parents on the two Llama 5% slices. Thus the simplest union does not provide the proposed complementarity at the tested settings.

KV quantization and attention engines. KIVI motivates asymmetric group-wise low-bit KV quantization and keeps residual tokens at full precision [7]. PriorityKV's INT4 path is not a new quantizer and is not positioned as a replacement for KIVI. It is a concrete storage path for testing role-aware precision placement and accounting actual payload bytes. FlashInfer exposes composable attention kernels and log-sum-exp state merging for heterogeneous KV layouts [8]. PriorityKV uses that merge contract, while retaining an explicit BF16 cold scratch limitation.

Long-context evaluation. LongBench and RULER test broad long-context capabilities [9, 10]. Our suite instead targets exact failure attribution in synthetic agent protocols. It cannot establish workload prevalence or replace general long-context suites.

Agent and tool-use evaluation. BFCL adds stateful multi-turn and multi-step function-calling tasks [11], while τ-bench evaluates policy-constrained interaction with tools and a simulated user [12]. We use BFCL for external task success and public τ-bench trajectories for a generation-free retention audit. Neither benchmark was used to construct PriorityBench-A.

3 PriorityBench-A

3.1 Tasks and scoring

PriorityBench-A generates multi-turn chats in which an early span contains the state required by a final request and intervening turns provide benign filler. Scoring is binary and programmatic; no LLM judge is used. Table 1 summarizes the three equally represented categories in the locked 240-example manifest.

Table 1: PriorityBench-A task families in the locked manifest ($n = 240$; 80 examples per category).

Category	Required behavior	Deterministic scorer
Tool schema	Emit a valid call under an early contract	JSON/schema subset
Instruction supersession	Follow the latest conflicting constraint	Required/forbidden regex
Multi-turn state	Reuse an earlier ID, path, or preference	Exact slot match

Two frozen manifests serve different studies. A 240-example set covers nominal 8k, 16k, and 32k contexts for mixed-precision quality; a 120-example stress set covers 8k and 16k contexts for retention. Both balance the three task families. Table 2 records their digests.

3.2 Placement and leakage controls

The generator's visible templates are also a threat: a heuristic tagger can appear semantic if gold state is consistently short, early, or marked by a template recognizable to the policy. We therefore evaluate two transformations of each stress slice. *Middle relocation* moves state away from the prefix while retaining the leading system message and final request. *Buried state* lengthens or embeds state-bearing turns so that a short-span heuristic does not protect all gold.

A separate CPU audit maps the gold-bearing messages into tokenizer spans and intersects them with the selected positions. This audit uses no model outputs. It measures the gold fraction already in sink plus recent and the gold fraction retained by each policy. Thus placement robustness and label leakage into the always-kept window are tested independently.

4 Method

4.1 Roles and matched-budget retention

Let a tokenized trace be $x_{1:n}$ with structural roles $r_{1:n}$. For keep fraction k, the nominal budget is

$$B(n,k) = \mathrm{round}(kn). \tag{1}$$

The mandatory set is $M = M_{\text{sink}} \cup M_{\text{recent}}$, where the first 16 tokens and last 128 tokens are protected. In the evaluated token-level regime, each compressed arm selects $A \supseteq M$ with

$$|A| = \max(B(n,k), |M|). \tag{2}$$

Position-blind retention uses the mandatory sink and fills from the recent edge. A corrected random control keeps a fixed M and samples the remaining budget; the released synthetic selector accidentally duplicates the position-blind indices, so those rows are collapsed in Section 6.1. Structure-aware retention first includes system, tool, constraint, and other state-bearing positions, then fills any residual budget from the recent edge. Whole-page experiments apply the same priority order while respecting page boundaries and never exceeding the token budget except for M.

The tagger consumes message roles and transparent schema or constraint markers. It does not inspect benchmark answers or future attention. The policy protects explicit protocol roles more reliably than unmarked free-form state; the buried controls in Section 6.1 quantify that boundary.

This ordering exposes a budget test. Let S denote structurally protected positions and retain the mandatory positional set M from above. If $|S \cup M| \le B$, every protected position fits and the structural score can separate it from filler. If $|S \cup M| > B$, the protected set is *oversubscribed*: a binary role label cannot order all protected positions, so the evaluated policy resolves the tie by position. We report the measured protected-role fraction as a cheap proxy for this condition; the fixed sink/recent contribution is explicit in M.

4.2 Pages and mixed precision

The physical page size is $P = 16$ tokens. In the mixed-precision study, every position remains present and an exact target fraction $f = 0.75$ is stored as INT4, subject to the common sink/recent BF16 protection. The position-blind arm spaces INT4 positions across the demotable region. The structure arm demotes filler, generated, and low-risk positions first, spilling into protected soft roles only if the byte budget requires it. Both arms therefore have the same number of INT4 positions.

Figure 3 traces the implemented path from tagging to the page table. It separates policy decisions—retention for eviction or dtype for mixed precision—from physical payload. Cold K and V pages use one uint8 container per 4-bit code plus per-group FP32 scale and zero-point metadata; hot pages remain BF16. Thus 0.719× is the measured implementation payload, whereas the 0.473× byte model assumes two codes per byte and lighter metadata.

4.3 Two-call FlashInfer decode

Prefill uses the native Hugging Face path. The resulting cache is partitioned into coalesced hot BF16 pages and uint8-coded cold pages. Before the frozen full-scratch decode measurement, cold pages are dequantized into BF16 GPU scratch. Each layer makes at most two homogeneous FlashInfer calls: one over hot pages plus the decode tail and one over cold scratch. Their outputs and native log-sum-exp states are combined with `merge_state`. We measured a maximum absolute error of approximately 4.88×10^{-4} after correcting an LSE-base mismatch.

Figure 4 shows why cache payload, allocator peak, and latency must be reported separately. The code-plus-metadata payload persists, full cold scratch is live during decode, and the decode tail grows token by token. A separate streamed-cold experiment instead dequantizes chunks and releases them after merging, but its three-example run is only a successful smoke test and is not a comparative systems result.

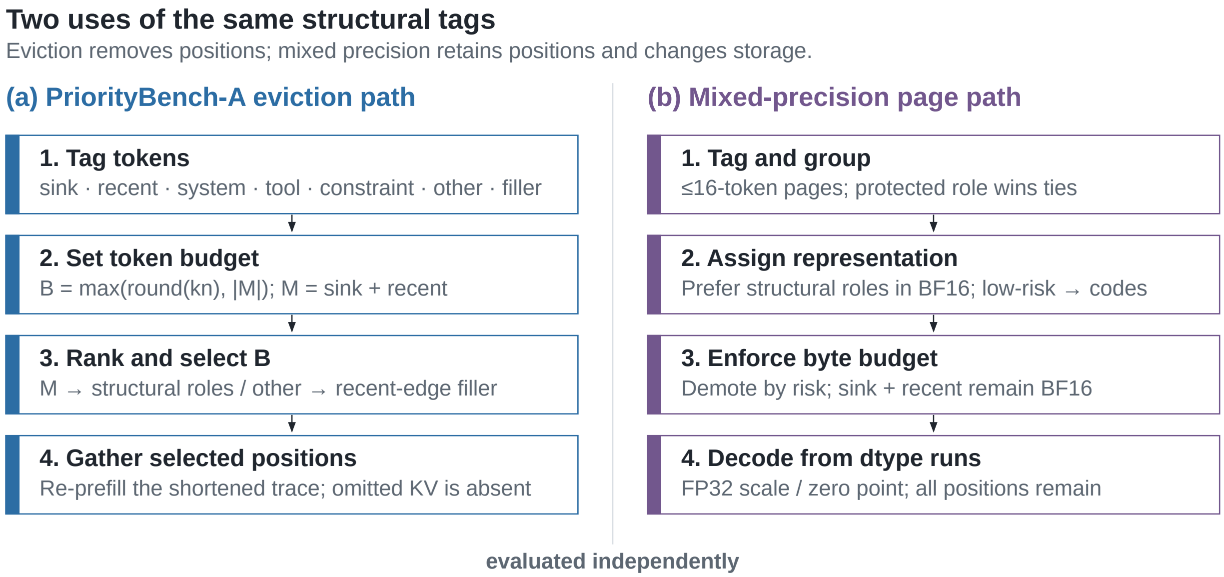

Figure 3: PriorityKV architecture. Chat metadata becomes token roles and pages of at most 16 tokens. Eviction and precision placement then use separate controllers and are evaluated independently.

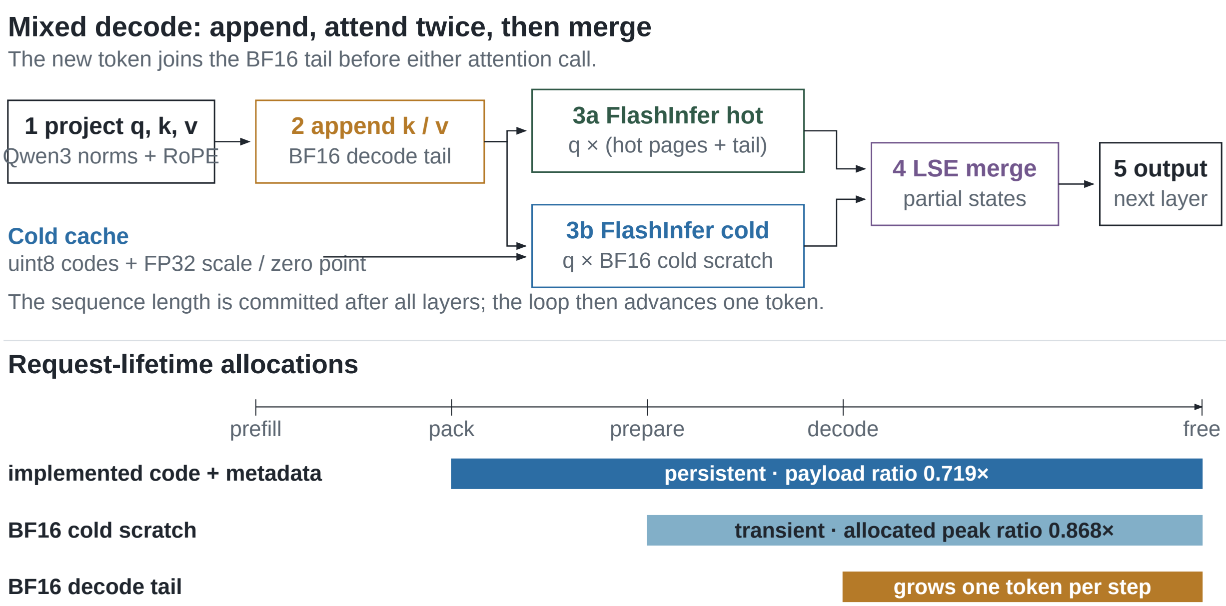

Figure 4: Decode path and allocation lifetime. Two homogeneous FlashInfer calls are merged through log-sum-exp state; the coded payload persists while BF16 cold scratch is live during decode.

5 Experimental Design

All GPU experiments use greedy decoding on NVIDIA H200 hardware. Canonical jobs use at most two GPUs, while each eviction, attention-baseline, and transfer job uses one. Table 2 records the pinned model revisions and manifest digests used throughout.

Table 2: Pinned model and manifest identity.

Item	Revision or digest
Qwen3-8B	`b968826d9c46dd6066d109eabc6255188de91218`
Llama-3.1-8B	`0e9e39f249a16976918f6564b8830bc894c89659`
Locked manifest (240)	SHA256 `fc44b966725738c94008ba61ce57ad7366169b9c0be73074f8161d909ccfae89`
Stress manifest (120)	SHA256 `154c07b900a7be6a26a0381b0fa28d49e0847086eade0b98e9472663b578784f`

5.1 Eviction, attention baselines, and placement controls

The eviction and placement study evaluates Qwen on the 120-example stress manifest, divided into three disjoint 40-example slices and balanced across 8k and 16k contexts. At token keep $k = 0.25$, it compares structure, the released position-blind selectors, keep-all, and a separately recorded FullKV run using deterministic task pass rate. The two released position-blind selectors are identical and are reported once. Middle-relocation and buried-state transformations are evaluated on all three slices. A second comparison reuses the same examples, model, contexts, and budget for SnapKV, PyramidKV, H2O, and a structure–SnapKV hybrid. SnapKV uses a 64-token observation window and kernel size five. The H2O arm is a chunked SDPA reimplementation and carries a fidelity limitation. We compute the exact two-sided paired McNemar test for structure versus SnapKV.

5.2 CPU retention audit

The audit runs the Qwen and Llama tokenizers on the three stress slices for each model ($n = 120$ each), maps gold messages to token spans, and applies position-blind and structure selection at $k = 0.25$. It reports mean token-retention fractions rather than model accuracy. These are descriptive span means, so no binomial interval or significance test is attached.

5.3 Mixed-precision quality and systems measurements

The locked mixed-precision study evaluates Qwen on all 240 examples: 83 at 8k, 81 at 16k, and 76 at 32k. FullKV, uniform-mixed, and structure-mixed arms use matched $f = 0.75$ INT4 placement for the mixed arms. The H200 mixed-cache FlashInfer path is scored by the same deterministic benchmark metric. This family tests a precision-placement quality hypothesis, not eviction, so we report Wilson intervals from the observed success counts rather than a paired test.

The latency measurement uses nine 8k and nine 16k examples, 128 generated tokens, one warm-up, and three timing repetitions summarized by per-example medians. FullKV uses SDPA while the mixed-cache path uses the FlashInfer shim; this is an end-to-end prototype comparison, not a kernel-isolated study. The memory measurement uses 18 examples and records coded payload plus allocated and reserved CUDA peak. Both are single-request studies without concurrent serving or tail latency. A three-example streamed-cold run checks exit status and parity but serves only as a smoke test.

5.4 Cross-model transfer

The Llama transfer study evaluates structure, SnapKV, H2O, PyramidKV, and hybrid on all three stress slices at $k = 0.25$ ($n = 120$). At $k = 0.05$, only the first two slices are run ($n = 40$ each); no uniform Llama arm is available.

6 Results

6.1 Structure protects state that blind eviction drops

At matched 25% token keep, structure passes 112/120 Qwen examples while the position-blind control passes 1/120 (Table 3). The corresponding Wilson intervals do not come close to overlapping. We report a single position-blind control rather than two: our released “random” arm is byte-identical to the uniform arm, a defect characterized in §9.1 rather than left for a reader to discover. Figure 5 places the position-blind and attention-derived selectors on one pass-rate axis.

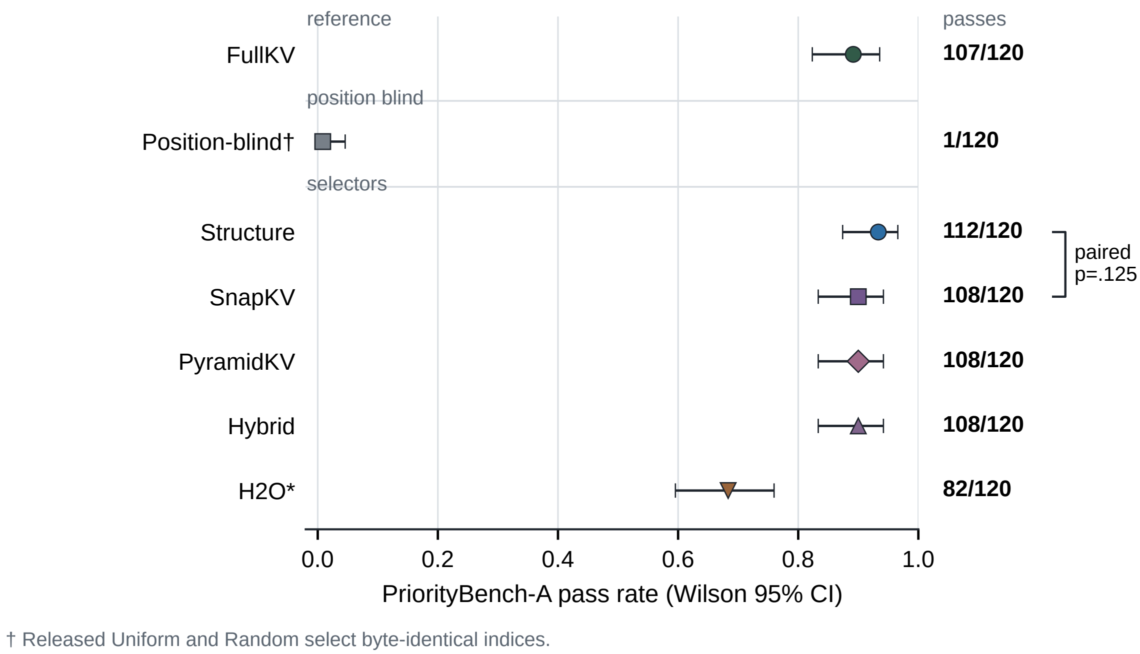

Figure 5: Matched-budget Qwen comparison ($n = 120$, $k = 0.25$). Points show pass rates with Wilson 95% intervals; exact counts are aligned at right and the bracket reports the paired structure–SnapKV McNemar test. H2O* denotes the chunked repository reimplementation.

Table 3: Pooled Qwen eviction and attention-baseline results at $k = 0.25$ ($n = 120$). Intervals are Wilson 95%.

Arm	Passes	Rate	Wilson 95% interval
FullKV	107/120	0.892	[0.823, 0.936]
Structure	112/120	0.933	[0.874, 0.966]
Position-blind	1/120	0.008	[0.002, 0.046]
SnapKV	108/120	0.900	[0.833, 0.942]
PyramidKV	108/120	0.900	[0.833, 0.942]
Hybrid	108/120	0.900	[0.833, 0.942]
H2O (chunked)	82/120	0.683	[0.596, 0.760]

The CPU audit confirms that this gap reflects retention rather than an easy always-kept window. Only 1.0–1.3% of Qwen gold tokens and none of the Llama gold tokens lie in sink plus recent (Table 4). Structure retains all mapped gold tokens, whereas uniform retains only the small fraction already in that window.

Placement controls bound the positive result (Table 5). Moving state to the middle gives both structure and FullKV 115/120, or 0.958 [0.906, 0.982]. Burying state in longer free-form turns lowers structure to 80/120, or 0.667 [0.578, 0.745], versus FullKV at 107/120, or 0.892 [0.823, 0.936]. Uniform scores 3/120 in the middle condition and 0/120 when state is buried. Visible roles are therefore a useful prior, not an oracle for arbitrarily embedded state.

Table 4: Gold-span retention audit at $k = 0.25$ ($n = 40$ per slice). Entries are mean token fractions, not pass counts.

Model	Slice	Gold in sink+recent	Structure kept	Uniform kept
Qwen	0	0.010	1.000	0.010
Qwen	1	0.013	1.000	0.013
Qwen	2	0.010	1.000	0.010
Llama	0	0.000	1.000	0.000
Llama	1	0.000	1.000	0.000
Llama	2	0.000	1.000	0.000

Table 5: Placement controls ($n = 40$ per slice), reported as structure / FullKV / uniform pass rates.

Slice	Middle relocation	Buried state
0	0.975 / 0.975 / 0.025	0.675 / 0.900 / 0.000
1	0.950 / 0.950 / 0.050	0.650 / 0.875 / 0.000
2	0.950 / 0.950 / 0.000	0.675 / 0.900 / 0.000

6.2 Structure reaches the attention-selector range in its sparse-state regime

PriorityBench-A establishes the sparse-state regime: the protected set fits inside the budget, and structure reaches the same measured range as attention selection. Section 7 probes the complementary schema-dense regime, where the structural score saturates. The protected fraction in Section 7.2 connects the two outcomes.

Structure passes 112/120, while SnapKV, PyramidKV, and hybrid each pass 108/120 (Table 3). The structure interval overlaps every 0.900 arm. The paired structure–SnapKV comparison has $b = 4$ cases where structure alone passes and $c = 0$ in the opposite direction; exact two-sided McNemar $p = 0.125$. The difference is not statistically significant. Hybrid equals SnapKV rather than exceeding it, so this simple union provides no additional benefit at the tested setting. H2O passes 82/120 using the chunked implementation; its implementation caveat prevents a reference-H2O claim.

6.3 Budget determines when the structural prior remains discriminative

All five tested structure and attention arms pass all 120 Llama examples at $k = 0.25$; the Wilson interval for 120/120 is [0.969, 1.000]. With no uniform arm, this is an easy-task ceiling rather than evidence of universal transfer. At $k = 0.05$, SnapKV passes 40/40 on each evaluated slice, while structure passes 35/40 and 36/40. No paired test was produced for these Llama slices.

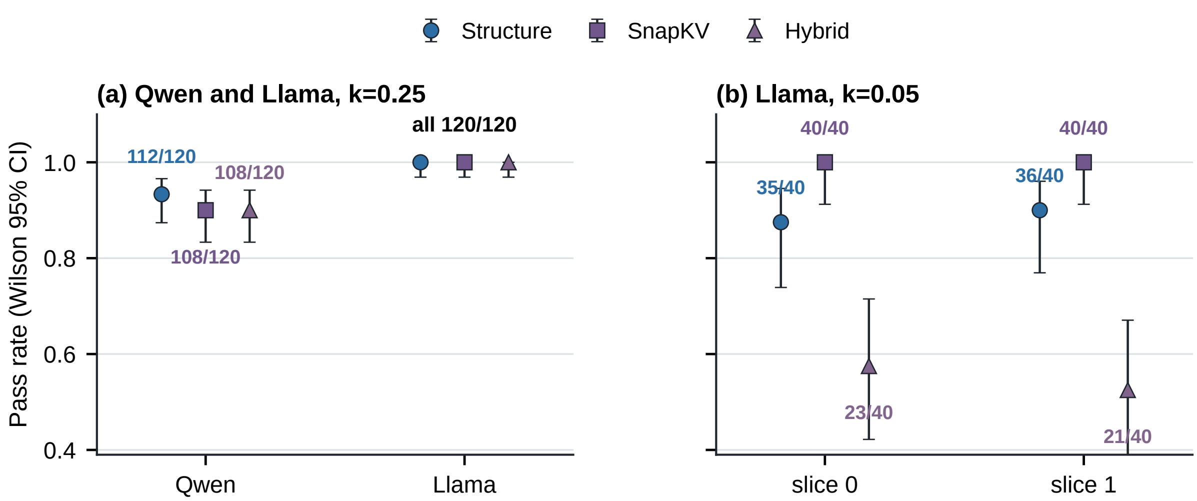

Figure 6: Budget and model dependence. Qwen and Llama at $k = 0.25$ use $n = 120$; each Llama $k = 0.05$ condition uses $n = 40$. Error bars are Wilson 95% intervals.

At the tighter budget, failures concentrate in tool-schema tasks. Structure passes 8/13 and 9/13 schema examples in the first and second slices, while SnapKV passes 13/13 in both. Hybrid is worse at 1/13 and 0/13, with overall rates 0.575 and 0.525. Protecting broad structural spans can therefore consume too much residual budget, and simply unioning role and attention selections does not make them complementary.

6.4 Role-aware and uniform INT4 placement preserve similar quality

On the locked 240 examples, FullKV passes 213, uniform-mixed 211, and structure-mixed 212 (Table 6). The structure–uniform difference is one example, and no paired statistical test supports a quality distinction. At 8k and 16k, every arm is 1.0; at 32k, FullKV, uniform, and structure are 49/76, 47/76, and 48/76. Figure 7 shows the count-backed Wilson intervals by context. The tested data do not support a role-aware INT4 quality advantage.

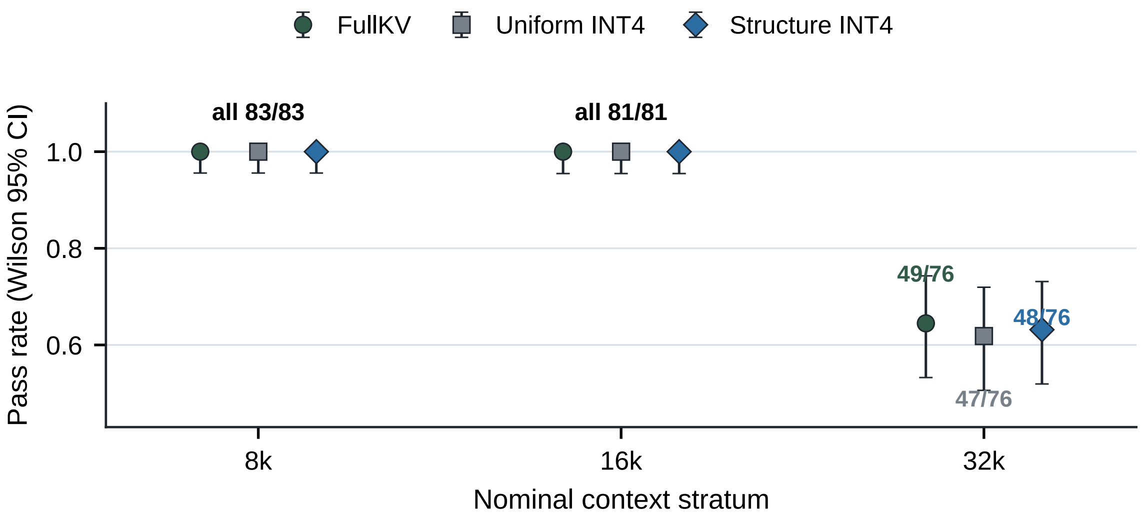

Figure 7: Matched INT4 quality. Qwen3-8B with FullKV or matched $f = 0.75$ INT4 placement. Error bars are Wilson 95% intervals from exact counts (83, 81, and 76 examples).

Table 6: Locked Qwen quality ($n = 240$). Intervals are Wilson 95%.

Arm	Passes	Rate	Wilson 95% interval
FullKV	213/240	0.8875	[0.841, 0.922]
Uniform mixed	211/240	0.8792	[0.832, 0.915]
Structure mixed	212/240	0.8833	[0.837, 0.918]

6.5 Storage savings incur scratch and latency costs

The mixed structure path has a measured code-plus-metadata payload ratio of 0.719× and an allocated-peak ratio of 0.868× FullKV on the 18-example memory slice. An idealized byte model gives 0.473× by assuming two INT4 codes per byte and lighter metadata; the current implementation instead stores each code in a uint8 container with FP32 scale and zero-point arrays. BF16 cold scratch further limits the peak-memory reduction. Figure 8 reports these memory and latency ratios. The canonical jobs were not independently repeated, so we do not show uncertainty bars.

At 8k, structure packing and scratch construction take 34.8 and 14.2 ms. E2E TTFT is 1.118× FullKV, and TPOT rises from 25.25 to 30.29 ms (1.200×). At 16k, pack and scratch take 48.1 and 19.7 ms; E2E is 1.113×, and TPOT rises from 23.84 to 28.95 ms (1.215×). This is the measured latency cost associated with the payload savings. The streamed-cold smoke run exits successfully with parity and approximately 36.4 GiB peak in its log, but with only three examples and no comparative frontier it remains a smoke test rather than a systems result.

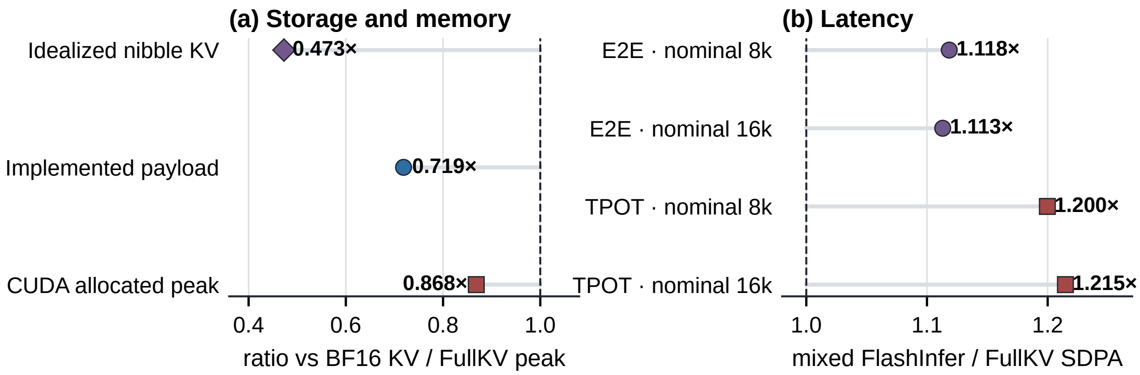

Figure 8: H200 systems trade-off. The implemented code-plus-metadata payload is smaller than FullKV KV, while the allocated CUDA peak falls less. Latency compares mixed FlashInfer against FullKV SDPA at nominal context strata; the dashed line marks parity.

7 External Evaluation: Finding the Boundary

PriorityBench-A shows that visible structure can protect sparse state. We next ask when that prior stops being informative. BFCL V3 multi-turn provides a sharply different, externally authored context distribution [11]. We score it with the unmodified official `multi_turn_checker`, which executes tool calls against stateful API implementations and compares final state and execution responses against ground truth. All non-FullKV arms are `kvpress` presses over an identical full prefill at an identical compression ratio, so the arms differ only in which KV entries survive, never in how they are removed.

7.1 BFCL exposes the high-protected-mass regime

We start with 150 conversations per model at a 25% keep budget. After paired context-limit exclusions, 140 Qwen and 143 Llama conversations remain (Table 7).

Table 7: BFCL V3 multi-turn, official scorer, 25% keep. Paired completeness 0.933 (Qwen) and 0.953 (Llama); exclusions are all `MODEL_CONTEXT_LIMIT`; zero matched-budget violations.

Arm	Qwen3-8B (n=140)	Llama-3.1-8B (n=143)
FullKV	0.193	0.077
SnapKV	0.136	0.084
ADAPT (§7.3)	0.129	—
Structure	0.000	0.000
Uniform	0.000	0.000
Random	0.000	0.000

The low FullKV scores, 0.193 and 0.077, make harness validation essential. Replayed ground truth passes the upstream checker, while corrupted arguments and dropped turns fail, and the zero-arm Wilson upper bound (0.027) sits below FullKV on both models. The reported runs use a corrected pipeline; §9.1 documents this fix alongside the other baseline correction.

This schema-dense regime locates the opposite end of the operating range: the binary structural score has no residual ranking signal and records 0/140 on Qwen and 0/143 on Llama. Exact paired McNemar gives FullKV versus structure $p = 1.5 \times 10^{-8}$ (Qwen) and $p = 9.8 \times 10^{-4}$ (Llama). Attention selection, by contrast, is not significantly different from FullKV at a 4× smaller cache: $p = 0.152$ on Qwen and $p = 1.0$ on Llama. Structure versus SnapKV is significant on both ($p = 3.8 \times 10^{-6}$ and $p = 4.9 \times 10^{-4}$). The central pattern—a hard structural prior exhausts its signal while attention remains useful—thus replicates across two model families.

7.2 Protected-set oversubscription explains the crossover

The crossover has a measurable mechanism. A binary role score separates protected tokens from filler only while the protected set fits inside the keep budget. Once oversubscribed, the evaluated policy must choose among equally protected tokens using its positional tie-break. Figure 9 measures the protected-role fraction with the same tagger used by the policy.

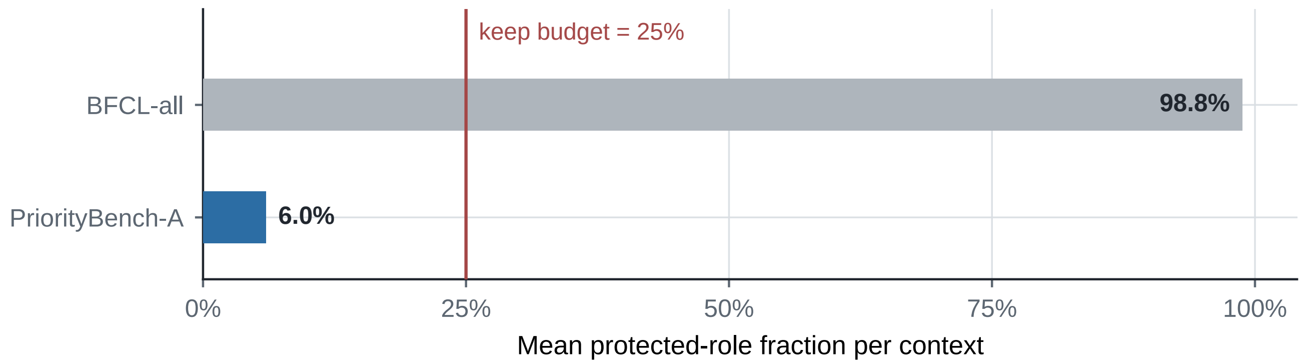

Figure 9: Protected mass relative to the keep budget. PriorityBench-A stress set has 6.0% mean protected-role mass ($n = 120$; 0/120 contexts exceed the budget), leaving room within a 25% budget. The sampled BFCL contexts are dominated by 32 JSON tool schemas and have 98.8% protected mass, so the role signal is oversubscribed. These two task-success workloads establish the observed crossover; broader validation remains future work.

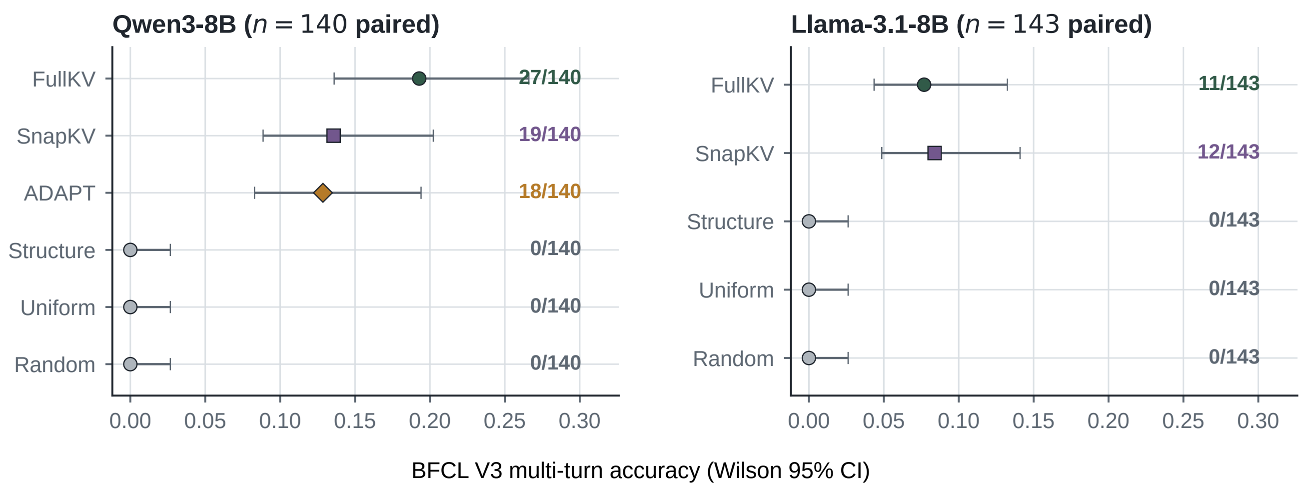

Figure 10: Paired BFCL accuracy with Wilson intervals. The same ordering holds on both models: no significant FullKV–SnapKV difference is detected, while the three non-attention policies are at zero with a Wilson upper bound of 0.027.

The two datasets therefore test different sides of the same rule. PriorityBench-A contains distractor filler by construction, the regime in which "protect the structure, drop the filler" is effective. The sampled BFCL prompts contain little analogous filler, so broad role protection cannot discriminate.

A generation-free audit of 4,856 public τ-bench trajectories (828,000 extracted spans) corroborates the mechanism from the other side: structure retains explicit policy lines at 0.820 versus 0.001 for a position-only budget, but loses on reused identifiers (0.055 versus 0.315). Structure wins precisely where protected spans are scarce and separable.

7.3 ADAPT softens the prior beyond the boundary

The hybrid evaluated in §6.3 force-protects structural positions, which consumes the entire budget whenever protected mass exceeds it. That is a mis-specification rather than a dead end. ADAPT replaces the hard constraint with a budget-relative prior,

$$\alpha = \min\left(1, \frac{\text{keep budget}}{\text{protected mass}}\right), \qquad s = \alpha \cdot \mathrm{rank}(s_{\text{struct}}) + (1-\alpha) \cdot \mathrm{rank}(s_{\text{attn}}), \tag{3}$$

where both quantities are known from the prompt and no scalar is fitted to task outcomes. Rank normalization makes the unlike score scales comparable. When $\alpha = 1$, ADAPT recovers the evaluated structure ordering exactly; as oversubscription grows, it assigns increasing weight to attention. It does not become pure SnapKV at the measured 25% boundary.

On BFCL the measured α is 0.267 on average (range 0.250–0.401) over 833 generation steps. ADAPT scores 0.129 versus 0.136 for SnapKV (exact McNemar $p = 1.0$, $\Delta = -0.007$, 95% CI $[-0.057, +0.071]$) and has no significant paired difference from FullKV ($p = 0.108$); it significantly exceeds the three non-attention policies ($p = 7.6 \times 10^{-6}$). This result shows recovery to the attention-based range on this workload, not superiority to SnapKV. The formula was chosen before the recorded ADAPT run but committed afterwards, so we treat the experiment as exploratory rather than preregistered evidence.

8 Discussion

The boundary turns structure into a deployable prior. Visible protocol structure protects sparse, recognizable state that positional selection drops. The middle and buried controls show what the label must capture; the Llama and BFCL results show what the budget must accommodate. The relevant test is not whether protected content is a majority, but whether $|S \cup M|$ exceeds the keep budget. This quantity is available before generation, so a serving policy can choose hard structural retention only when the prior can discriminate.

Soft weighting is safer than a hard union. The original structure–SnapKV union equals SnapKV on Qwen and falls below both parents on Llama at 5%, because protected positions can consume the residual budget. ADAPT addresses that mechanism directly: it preserves the structural ordering when the set fits and shifts weight toward attention as oversubscription grows. Its BFCL result supports this recovery mechanism, while remaining exploratory and limited to one budget.

Bytes, peak, and speed are different systems quantities. Uint8-coded values and scale metadata reduce payload. Full-layer BF16 cold scratch then expands the live working set and adds preparation and decode cost. The observed 0.719× payload, 0.868× peak, and roughly 1.20× TPOT are mutually consistent. They do not establish serving throughput or capacity under concurrency.

9 Limitations and Threats to Validity

Synthetic workload. PriorityBench-A is generated from controlled templates and exact scorers. It measures whether particular protocol state survives, not how often these failures occur in deployed traffic. Its 6.0% mean protected-role mass places it at one end of the measured boundary; BFCL tests the other end.

External evaluation scope. The external results cover one budget, two 8B models, and 150 starting conversations per model. FullKV scores 0.193 and 0.077, leaving limited absolute headroom even though the arms separate. The τ-bench audit measures retention only—no simulator, backend, or generation—and excludes SnapKV, whose scores require realized attention.

Tagger–generator alignment. The tagger uses role, length, and schema/constraint markers that resemble the generator's visible templates. The gold audit rules out answer labels already in sink plus recent, but it does not eliminate all template alignment. The buried result—0.675, 0.650, and 0.675 versus FullKV 0.900, 0.875, and 0.900—is the clearest measured boundary.

9.1 Corrections

Two implementation bugs were caught and fixed before the reported runs; we document both here rather than leave a reader to find them. First, the released synthetic study listed "uniform" and "random" as separate controls, but `select_random` expands its forced recent window until the budget is full, leaving nothing to sample. We verified byte-identical index sets at $n \in \{600, 4096, 40000\}$ and therefore collapse the two 1/120 columns into one position-blind result; no measured score changes. The BFCL study uses a corrected random arm with a fixed recent window and independently scores 0.000. Second, an early BFCL run mis-decoded reasoning blocks, which forced every arm, including FullKV, to zero; the BFCL results in §7 use the pipeline after that fix.

Baseline fidelity. SnapKV, PyramidKV, H2O, and hybrid are matched-budget repository reimplementations, not reference-code leaderboard runs; chunked H2O is the least faithful approximation.

Statistical scope. The paired Qwen structure–SnapKV result is not significant ($p = 0.125$), and the Llama 5% comparison has no paired test. Canonical runs use fixed example sets rather than machine replications; Wilson intervals describe pass counts, not hardware variability.

Model and budget scope. Qwen3-8B on H200 is the primary matrix. Llama provides a ceiling and a tighter-budget crossover, not universal transfer. Only selected 5% and 25% token budgets are tested.

Systems scope. The latency comparison uses different backends (FullKV SDPA and mixed FlashInfer) and single-request measurements. It does not establish batched throughput, concurrency, or tail latency. Payload is not peak memory because cold attention expands the coded cache into BF16 scratch.

10 Conclusion

PriorityKV turns application-visible structure into a retention rule with a known operating range: strong when protocol state is sparse and recognizable (112/120 versus 1/120 for position-blind eviction), and blunt once that state saturates the budget, as it does on BFCL. The practical payoff is the boundary itself: protected mass versus keep budget is measurable before a single token is generated, so a serving system can choose hard structural retention when it will work and fall back toward attention, via ADAPT, when it will not. The mixed-precision study applies the same honesty to storage: a real payload reduction at a measured, non-zero latency cost. In operational terms, retain the schema before the pleasantry, but check first that the budget can hold the schema.

References

[1] Ashish Vaswani et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[2] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. doi:10.1145/3600006.3613165.

[3] Guangxuan Xiao et al. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. OpenReview:NG7sS51zVF.

[4] Zhenyu Zhang et al. H2O: Heavy-Hitter Oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2023.

[5] Yuhong Li et al. SnapKV: LLM knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37, 2024. doi:10.52202/079017-0722.

[6] Zefan Cai et al. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. arXiv:2406.02069, 2024.

[7] Zirui Liu et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, PMLR 235:32332–32344, 2024.

[8] Zihao Ye et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving. *Proceedings of Machine Learning and Systems*, 7, 2025.

[9] Yushi Bai et al. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3119–3137, 2024. doi:10.18653/v1/2024.acl-long.172.

[10] Cheng-Ping Hsieh et al. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling (COLM)*, 2024. OpenReview:kIoBbc76Sy.

[11] Shishir G. Patil et al. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, PMLR 267:48371–48392, 2025. PMLR:patil25a.

[12] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.12045.